

DEPLOYMENT HANDBOOK

HCLS Agentic AI Suite

Deploying an Agent on AWS

Step-by-step, console and CLI — from an empty AWS account to a running, governed, human-gated agent. Worked example: Agent 02 — Pharmacovigilance, with a per-agent appendix for all eight.

Systems-Integrator field & delivery reference · June 2026

Illustrative reference — figures are mockups, not actual AWS screenshots. Validate for your environment.

Audience: the SI delivery engineer / cloud architect standing up one agent in a customer's AWS account for the first time. This is the "do exactly this" book. It covers both the **AWS Console** (click path) and the **CLI** equivalent for every step, from an empty account to a running, governed, human-gated agent.

Worked example: Agent 02 — Pharmacovigilance (ICSR Case Intake). Every step generalizes to the other seven agents via the **Per-Agent Appendix** at the end — only the `AgentId`, the systems you connect, the high-risk tools, and the approver role change.

0. How to use this handbook

There are two deployment shapes; the handbook walks the **native** shape end to end (it's the higher-fidelity, fully-managed target) and notes the container differences inline.

Shape	What runs	When to choose
Container lift	The LangGraph agent image on Bedrock AgentCore Runtime (or ECS Fargate)	Fastest; you want the agent code unchanged
Native rebuild (this book)	Deterministic steps in Lambda , drafting via Strands/Bedrock , orchestrated by Step Functions with a <code>waitForTaskToken</code> human gate	Highest fidelity to managed-serverless; clearest audit & HITL story

Both deploy through the **same CloudFormation quickstart** (`infra/cloudformation/quickstart.yaml`) — `DeployMode=native` vs `DeployMode=container`. Both put every system-of-record call behind the **MCP authorization gateway = Bedrock AgentCore Gateway + Identity**.

Time budget: ~2–4 hours for a first dev deployment once prerequisites are granted.

Polished PDF leave-behind: a print-ready, branded PDF of this handbook (cover + all figures) is at `../HCLS-Deployment-Handbook.pdf`.

Console mockups: labeled, illustrative AWS Console screens for the key steps are embedded inline (Figures 1–7) and stored in `docs/assets/console/` (PNG + SVG source).

Two ways to do Phase 1 (account foundation): the quickstart CloudFormation creates KMS, the Bedrock Guardrail, Cognito, IAM, audit/WORM storage **for you**. This book shows the **manual console steps too**, so you understand (and can demo to a customer's security team) exactly what each control is. If you trust the CFN, you can skip the manual sub-steps marked *(CFN does this)* and jump to Phase 2.

1. Roles & prerequisites

You need	Who provides it	Notes
An AWS account / sandbox with admin (or the scoped deploy policy)	Customer cloud team	Dev environment first; never start in prod
Amazon Bedrock model access for Claude	Account admin	One-time per account+region (Phase 3.1)
A Region that supports Bedrock AgentCore	Architect	e.g. <code>us-east-1</code> ; confirm AgentCore availability before you start
Enterprise IdP metadata (Okta/Entra/AD)	Customer identity team	SAML/OIDC metadata URL for Cognito federation
The system-of-record endpoint(s) + an API token	Customer system owner (e.g. PV/Safety team)	For Agent 02: the Argus/Veeva Safety REST base URL + token. You can deploy against the bundled reference service first.
Local tooling	You	AWS CLI v2, Docker or Finch (ARM64 builds), Python 3.11, <code>zip</code>

Decision rights to confirm before you touch the console: who is the named **approver role** for this agent's high-risk actions (Agent 02 = `PV_MEDICAL_REVIEWER`), and who signs the **computer-system validation** at go-live.

2. Pre-flight (local machine)

```
aws --version           # v2.x
aws configure           # set keys + default region (use an AgentCore
region)
aws sts get-caller-identity # confirm the right account
python3 --version       # 3.11+
docker version          # (container path only) confirm buildx/ARM64
```

Clone the repo and confirm it runs offline first — this is your fallback demo and your sanity check that the agent logic is healthy before any cloud spend:

```
cd 02-pharmacovigilance-agent
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && pip install -e ../platform_core
EXTRACT_MODE=demo pytest tests/ -q          # expect: all pass
EXTRACT_MODE=demo PYTHONPATH=../platform_core python demo/demo_live.py # full run,
local
```

3. Phase 1 — Account foundation

3.1 Enable Bedrock model access (Console)

This must be done once per account+region or every inference call fails with AccessDenied.

1. Sign in to the **AWS Management Console**; set the Region (top-right) to your target Region.
2. Search for **Bedrock** → open **Amazon Bedrock**.
3. Left nav → **Model access** → **Manage model access** (or **Modify model access**).
4. Check the **Anthropic – Claude** models you intend to use (the narrative + fast tiers).
5. **Save / Request access**. Access for Anthropic models is typically granted immediately; wait until status shows **Access granted**.

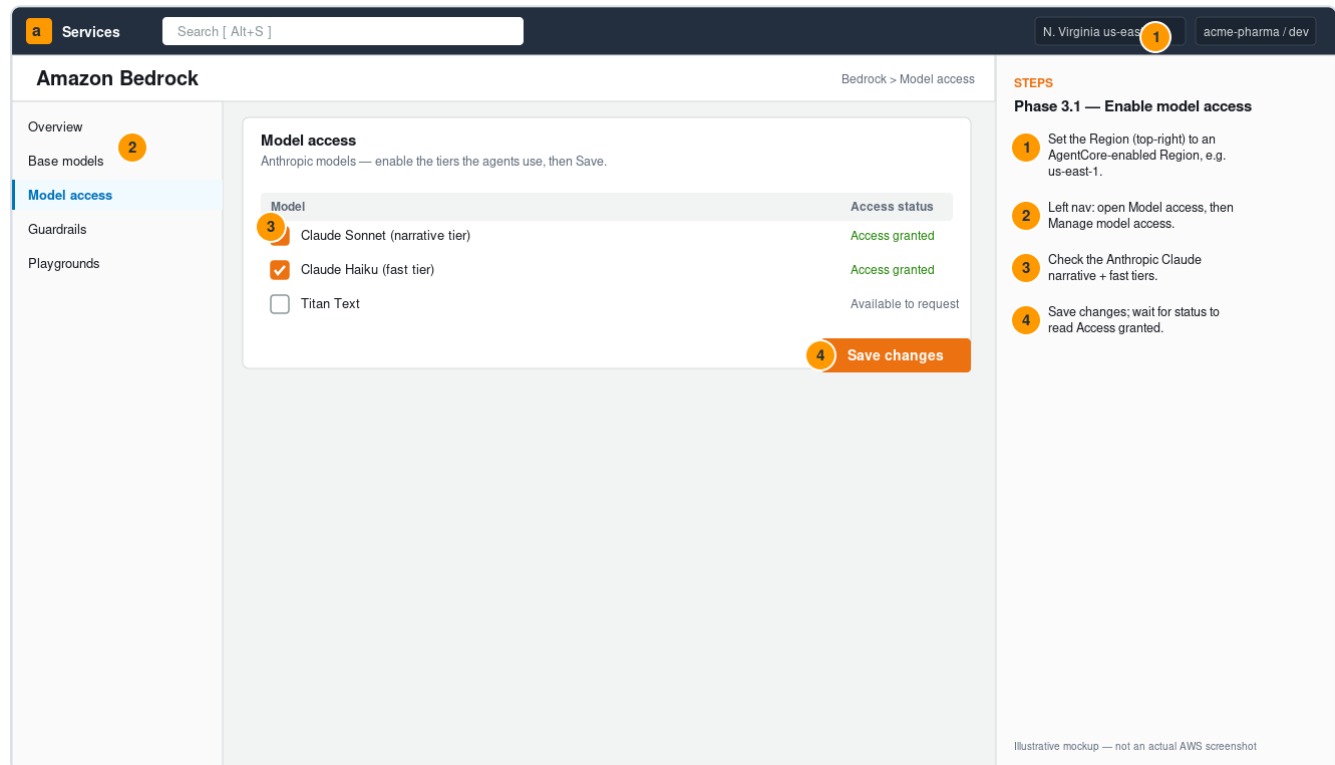


Figure 1. Amazon Bedrock → Model access: enable the Claude narrative + fast tiers (illustrative mockup, not an actual AWS screenshot).

CLI check:

```
aws bedrock list-foundation-models --region us-east-1 \
  --query "modelSummaries[?contains(modelId,'anthropic.claude')].modelId" --output
table
```

3.2 Create a Bedrock Guardrail (CFN does this — `security.yaml`)

The Guardrail is your PHI + off-label safety net on every inference. The quickstart creates one named `hcls-<env>-guardrail` . To create/inspect manually in the Console:

1. **Bedrock** → left nav **Guardrails** → **Create guardrail**.
2. Name it `hcls-dev-guardrail` . Add a **Sensitive information filter**: set **US SSN** → **Block**, **Email/Name/Phone** → **Anonymize**.
3. Add a **Denied topic** "OffLabelPromotion" (definition: promotion for an unapproved indication or absolute efficacy/safety claims).
4. **Create**, then **Create version** (note the **Guardrail ID** and **Version** — you'll pass the ID to the stack).

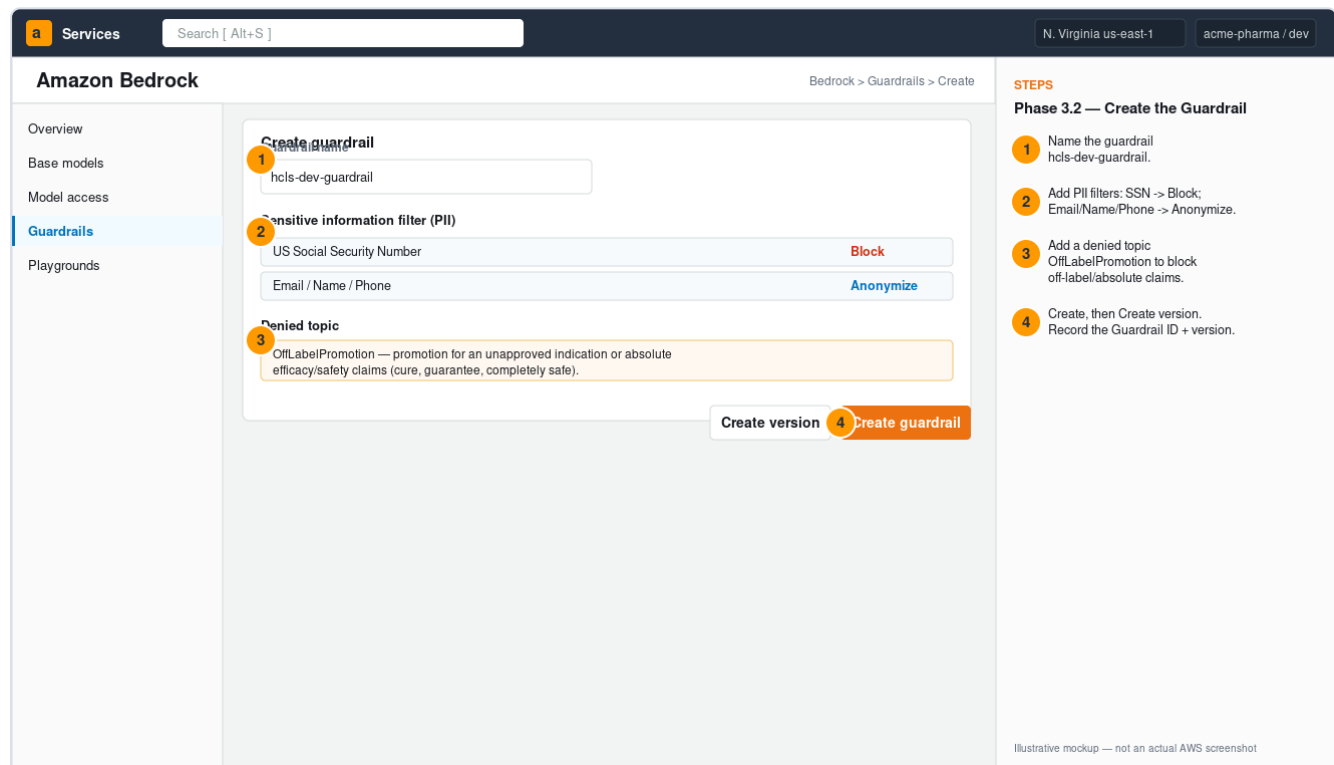


Figure 2. Bedrock → Guardrails: PHI filters + off-label denied topic (illustrative mockup, not an actual AWS screenshot).

3.3 Cognito user pool + IdP federation (CFN does this — `security.yaml`)

Identity is how the gateway knows *who* is acting and whether they may approve. The quickstart creates the pool; you connect it to the customer IdP:

1. **Amazon Cognito** → **User pools** → open `hcls-dev-users` (created by the stack).
2. **Sign-in experience** → **Federated identity provider sign-in** → **Add identity provider** → **SAML** (or OIDC).
3. Paste the customer's **metadata document URL** (the `IdpMetadataUrl` you'll also pass to the stack).
4. **Attribute mapping**: map the IdP group/claim that designates the approver to the custom attribute `custom:hcls_role` (e.g. group `GRP-PV-PHYSICIANS` → `PV_MEDICAL_REVIEWER`).

5. Save. Provisioning a user = adding them to the IdP group; no accounts are managed in AWS.

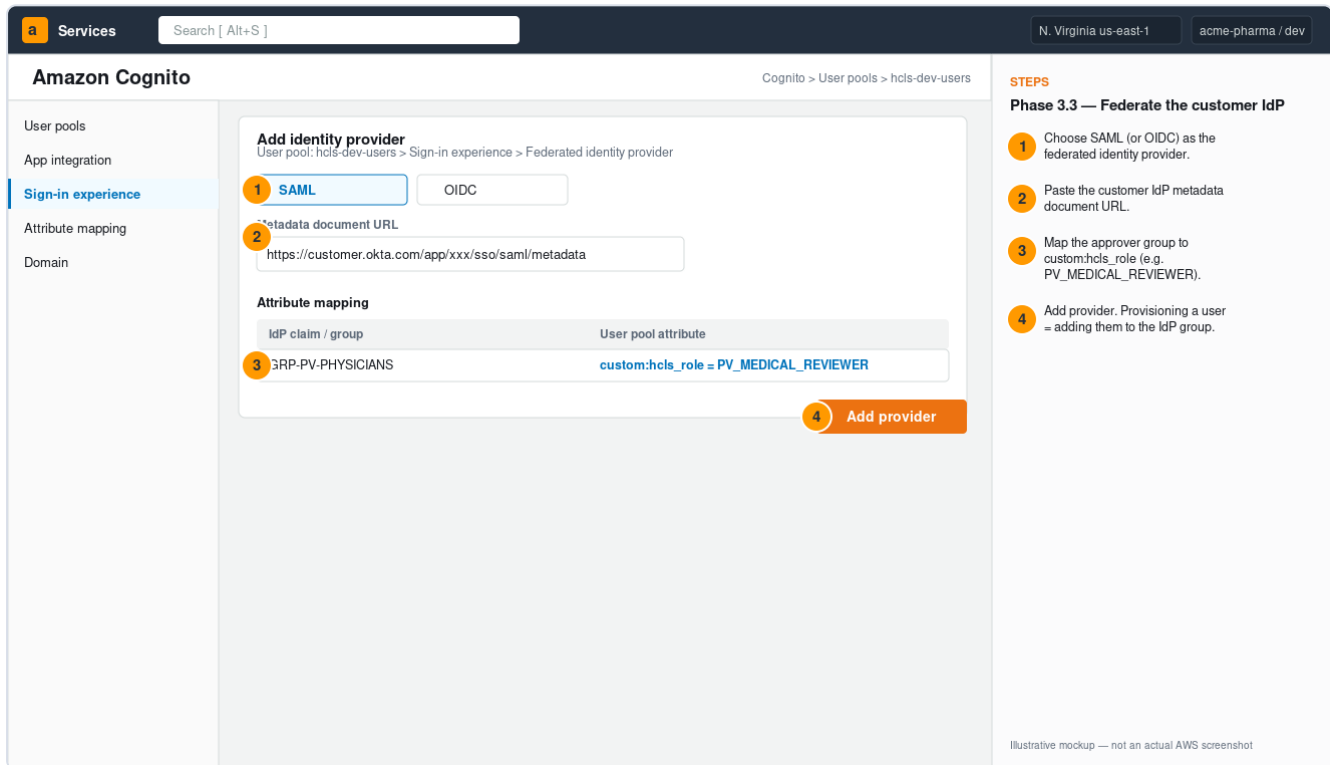


Figure 3. Cognito → Sign-in experience: federate the IdP and map the approver to custom:hcls_role (illustrative mockup, not an actual AWS screenshot).

3.4 KMS key (CFN does this — security.yaml)

The stack creates a per-environment CMK (alias/hcls-dev) with rotation enabled and uses it to encrypt the audit table, review table, and WORM bucket. No manual step needed unless you are bringing your own key (then note its ARN).

4. Phase 2 — Stage the artifacts

CloudFormation needs (a) the component templates in S3 and (b) the agent's Lambda code in S3. The container path also needs an image in ECR.

4.1 Create an S3 bucket and upload the templates (Console + CLI)

Console: **S3** → **Create bucket** → name `my-cfn-bucket` (Block Public Access ON) → Create. Then upload, or via CLI:

```
aws s3 mb s3://my-cfn-bucket
aws s3 cp infra/cloudformation/ s3://my-cfn-bucket/hcls/ --recursive --exclude "*" --include "*.yaml"
```

4.2 Package the agent Lambda code (native path)

```
cd aws-native-reference/02-pharmacovigilance
mkdir -p build
zip -r build/lambda.zip core.py strands_agent.py lambda/ requirements.txt
aws s3 mb s3://my-code-bucket
aws s3 cp build/lambda.zip s3://my-code-bucket/02-pharmacovigilance/lambda.zip
```

4.3 (Container path only) Build & push the ARM64 image to ECR

Console: **ECR** → **Create repository** → `hcls-02-pharmacovigilance`. Then:

```
ACCOUNT=$(aws sts get-caller-identity --query Account --output text); REGION=us-east-1
aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin $ACCOUNT.dkr.ecr.$REGION.amazonaws.com
# build from the shared AgentCore runtime contract (ARM64, /invocations + /ping, port 8080)
docker buildx build --platform linux/arm64 -t hcls-02 -f aws-native-reference/_shared/runtime/Dockerfile . --load
docker tag hcls-02:latest $ACCOUNT.dkr.ecr.$REGION.amazonaws.com/hcls-02-pharmacovigilance:latest
docker push $ACCOUNT.dkr.ecr.$REGION.amazonaws.com/hcls-02-pharmacovigilance:latest
```

5. Phase 3 — Deploy the infrastructure (CloudFormation)

This single stack provisions the customer-isolated environment: VPC + private subnets, KMS + Guardrail + Cognito + IAM (security), append-only DynamoDB audit + S3 Object Lock WORM + HITL review table (data), the **AgentCore Gateway + Identity** (MCP layer), and the per-agent service (Step Functions + Lambdas, or AgentCore Runtime).

5.1 Console (click path)

1. **CloudFormation** → **Create stack** → **With new resources (standard)**.
2. **Specify template** → **Amazon S3 URL** → `https://my-cfn-bucket.s3.amazonaws.com/hcls/quickstart.yaml` → **Next**.
3. **Stack name:** `hcls-dev-02-pharmacovigilance`.
4. **Parameters:** - `Environment = dev` - `AgentId = 02-pharmacovigilance` - `DeployMode = native` (or `container`) - `TemplateBaseUrl = https://my-cfn-bucket.s3.amazonaws.com/hcls` - `LambdaCodeBucket = my-code-bucket` - `LambdaCodeKey = 02-pharmacovigilance/lambdas.zip` - `IdpMetadataUrl` = the customer IdP metadata URL (optional in dev)
5. **Next** → leave defaults → **Next**.
6. **Capabilities:** check **"I acknowledge that AWS CloudFormation might create IAM resources with custom names."**
7. **Submit.** Watch the **Events** tab until **CREATE_COMPLETE** (~10–20 min; nested stacks for network/security/data/gateway/agent appear under **Stacks**).
8. Open the **Outputs** tab — record `GatewayId`, `GuardrailId`, `AuditTable`.

The image is a mockup of the AWS CloudFormation 'Create stack' console. It features a dark header with 'Services' and a search bar. The main content area is titled 'AWS CloudFormation' and 'CloudFormation > Create stack'. On the left, a sidebar lists 'Stacks', 'Create stack' (highlighted), 'StackSets', 'Exports', and 'Designer'. The central form is divided into sections: 'Amazon S3 template URL' with a text input containing 'https://my-cfn-bucket.s3.amazonaws.com/hcls/quickstart.yaml'; 'Stack name' with a text input containing 'hcls-dev-02-pharmacovigilance'; 'Parameters' with a table of key-value pairs; and an acknowledgment checkbox. The parameters table has the following data:

Parameter	Value
Environment	dev
AgentId	02-pharmacovigilance
DeployMode	native
LambdaCodeBucket	my-code-bucket
IdpMetadataUrl	https://customer.okta.com/.../metadata

Below the parameters is a checkbox labeled 'I acknowledge that AWS CloudFormation might create IAM resources with custom names.' which is checked. A 'Submit' button is at the bottom right of the form. On the right side, a 'STEPS' section titled 'Phase 3 - Deploy with CloudFormation' lists four steps: 1. Specify template, 2. Set parameters, 3. Check IAM capability, and 4. Submit. A footer note states 'Illustrative mockup — not an actual AWS screenshot'.

Figure 4. CloudFormation → Create stack: template URL, parameters, IAM capability, Submit (illustrative mockup, not an actual AWS screenshot).

5.2 CLI equivalent

```
aws cloudformation deploy \
  --template-file infra/cloudformation/quickstart.yaml \
  --stack-name hcls-dev-02-pharmacovigilance \
  --capabilities CAPABILITY_NAMED_IAM \
  --parameter-overrides \
    Environment=dev AgentId=02-pharmacovigilance DeployMode=native \
    TemplateBaseUrl=https://my-cfn-bucket.s3.amazonaws.com/hcls \
    LambdaCodeBucket=my-code-bucket LambdaCodeKey=02-pharmacovigilance/lambda.zip \
    IdpMetadataUrl=https://customer.okta.com/app/xxx/sso/saml/metadata
aws cloudformation describe-stacks --stack-name hcls-dev-02-pharmacovigilance \
  --query "Stacks[0].Outputs" --output table
```

Set `Environment=prod` in production: it forces the Bedrock Guardrail to be mandatory (the LLM factory refuses to start un-guardrailed) and turns on the stricter posture.

6. Phase 4 — The MCP layer (AgentCore Gateway + Identity)

The `agentcore-gateway.yaml` nested stack created the **Gateway** (JWT-authorized by your Cognito pool) and one **target per system of record**. To see/extend it in the Console:

1. **Bedrock** → **AgentCore** → **Gateways** → open `hcls-dev-gateway`.
2. Confirm the **authorizer** points at your Cognito user pool discovery URL.
3. Under **Targets**, confirm the systems this agent uses exist (Agent 02: `safety`, plus `meddra / whodrug` coding). Each target maps to a tool in `platform_core/.../mcp_gateway/policy.py:TOOL_REGISTRY`.
4. High-risk targets (`safety.write_case_draft`, `safety.submit_report`) require a human approval token — this is enforced by the workflow's `waitForTaskToken` gate (Phase 7).

The deny-by-default authorization (agent grant $\cap$ user entitlement), scoped tokens, and audit semantics are identical to the Python reference in `platform_core` — useful when walking a customer's security team through exactly what the gateway does.

7. Phase 5 — Configure the agent (connect real systems)

7.1 Store the system-of-record credential (Secrets Manager — Console)

1. **AWS Secrets Manager** → **Store a new secret** → **Other type of secret**.
2. Key/value: key `safety_api_token` , value = the customer's Safety API token.
3. **Secret name:** `hcls/safety_api_token` (the `hcls/` prefix is what `get_secret` resolves).
4. Store. (The agent's IAM role already has `secretsmanager:GetSecretValue` on `hcls/*`.)

Figure 5. Secrets Manager → Store a new secret as `hcls/safety_api_token` (illustrative mockup, not an actual AWS screenshot).

7.2 Point the agent at live inference + live systems

For the native path these are Lambda environment variables (set by the stack from your parameters; override in the Lambda console under **Configuration** → **Environment variables** if needed):

Variable	Value	Effect
LLM_PROVIDER	bedrock	In-account inference (no data egress)
BEDROCK_GUARDRAIL_ID	from stack Outputs	PHI + off-label filter on every call
BEDROCK_REGION	e.g. us-east-1	
CONNECTOR_MODE	live	Use real connectors, not fixtures
SAFETY_BASE_URL	the Argus/Veeva REST base URL	Where LiveSafetyConnector calls
ENVIRONMENT	prod (in prod)	Makes the Guardrail mandatory

Tip: deploy first with SAFETY_BASE_URL pointed at the bundled reference service (02-pharmacovigilance-agent/demo/reference_safety_service.py , run on an internal host) to validate the whole pipeline before integrating the customer's real safety gateway. Swapping to the real endpoint is a one-variable change — no code change.

8. Phase 6 — Smoke test the workflow + the human gate

8.1 Start an execution (Step Functions — Console)

1. **Step Functions** → **State machines** → open `hcls-dev-02-pharmacovigilance`.
2. **Start execution.** Paste the contents of `aws-native-reference/02-pharmacovigilance/sample_input.json` as input → **Start execution.**
3. Watch the **Graph view**: `Assemble` → `Draft` → `Check` → `RouteChoice` → `HumanReviewGate`.
4. The execution **pauses at HumanReviewGate** — this is the `waitForTaskToken` HITL gate. It is *supposed* to wait. Nothing finalizes until a qualified human approves.

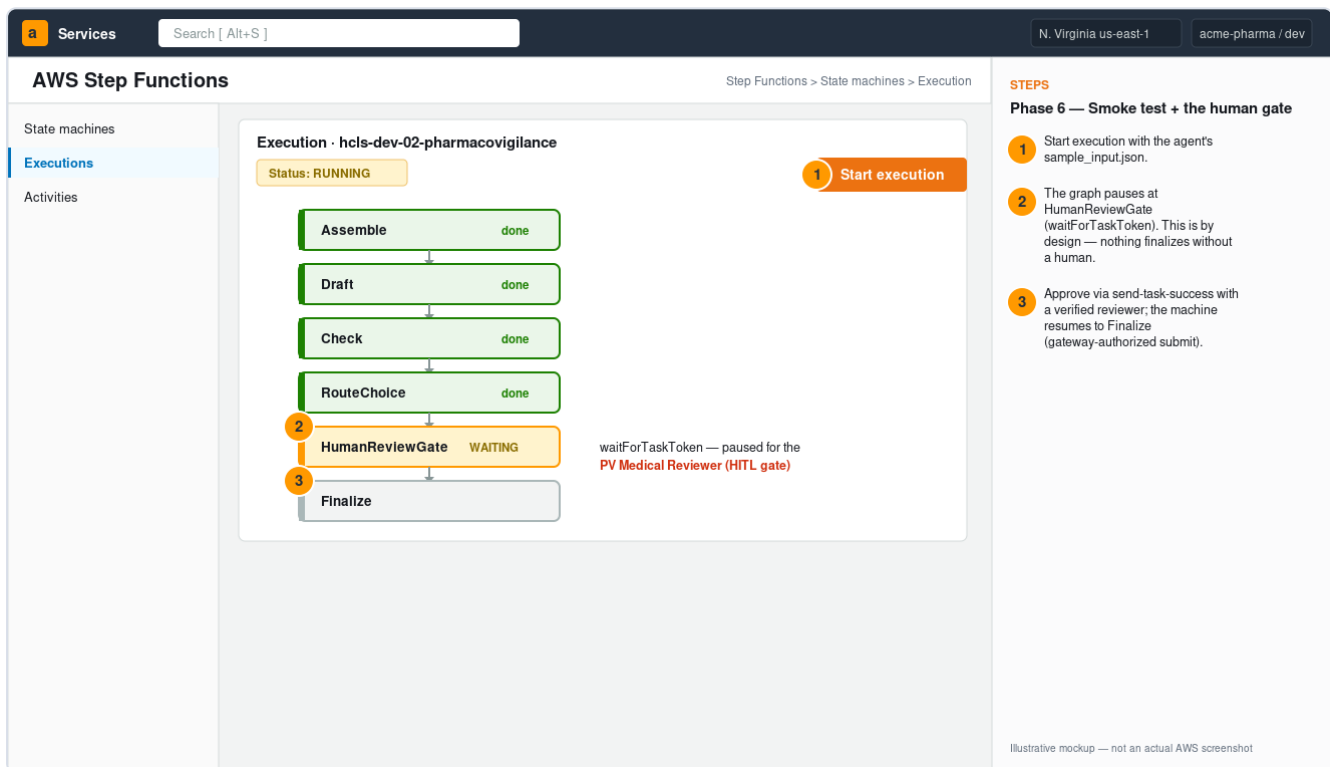


Figure 6. Step Functions execution paused at HumanReviewGate (waitForTaskToken HITL gate) (illustrative mockup, not an actual AWS screenshot).

CLI:

```
SM_ARN=$(aws cloudformation describe-stacks --stack-name hcls-dev-02-pharmacovigilance \
  --query "Stacks[0].Outputs[?contains(OutputKey,'StateMachine')].OutputValue" --output
text)
aws stepfunctions start-execution --state-machine-arn "$SM_ARN" \
  --input file://aws-native-reference/02-pharmacovigilance/sample_input.json
```

8.2 Approve as the PV Medical Reviewer (the human gate)

The `hitl_notify` Lambda wrote the **task token** + draft to the review table (the review UI reads it for the reviewer). To approve, the reviewer (or you, in dev) sends task success with their **verified identity** bound into the record:

```
aws stepfunctions send-task-success \
  --task-token "<token-from-review-table>" \
  --task-output '{"approved": true, "reviewer": {"sub": "pv-physician-1",
"custom:hcls_role": "PV_MEDICAL_REVIEWER"}}'
```

The state machine resumes to **Finalize**, which performs the gateway-authorized `safety.submit_report` and seals the audit trail. To **reject**, use `send-task-failure`.

8.3 Confirm the audit trail (DynamoDB — Console)

1. **DynamoDB** → **Tables** → open `hcls-dev-audit` → **Explore table items**.
2. You should see append-only entries for each step with PHI masked, the approver bound to the submit, and lineage to the safety system. This is your 21 CFR Part 11 evidence.

Amazon DynamoDB DynamoDB > Tables > hcls-dev-audit > Explore items

Tables

Explore items

PartiQL editor

Backups

Explore items · hcls-dev-audit

ts	ion	tool	user	args (PHI-masked)
16:31:02	ALLOW	safety.get_case	u-pv-1	case_id=ICSR-2026-0002
16:31:05	ALLOW	meddra.code_term	u-pv-1	term=[REDACTED]
16:31:09	PENDING_APPROVAL	safety.submit_report	u-pv-1	awaiting reviewer
16:34:20	ALLOW	safety.submit_report	u-md-7	approved_by=u-md-7

Append-only by IAM policy (UpdateItem/DeleteItem denied) — tamper-evident, 21 CFR Part 11.

STEPS

Phase 6.3 — Confirm the audit trail

- 1 Each workflow step writes an append-only audit item with PHI masked.
- 2 Note **PENDING_APPROVAL** on the high-risk submit before the human approves.
- 3 After approval the submit is **ALLOW** with `approved_by` bound to the reviewer.

Illustrative mockup — not an actual AWS screenshot

Figure 7. DynamoDB → Explore items: append-only, PHI-masked audit trail (21 CFR Part 11) (illustrative mockup, not an actual AWS screenshot).

9. Phase 7 — The reviewer experience (optional UI)

- **Container/ECS path:** the CloudFront → ALB (Cognito auth) → ECS Fargate Streamlit UI is the reviewer's screen; sign in via the federated IdP and approve from the dashboard.
 - **Native path:** wire the review table to the customer's existing work-queue UI, or run the bundled Streamlit `app.py` against the review table for a demo. The approval action calls `send-task-success` under the hood with the reviewer's Cognito identity.
-

10. Phase 8 — Validation & go-live checklist

Do not promote to prod until every box is checked:

- ☐ Bedrock model access granted; Guardrail attached and **tested** (submit a PHI string, confirm it's blocked/anonymized).
 - ☐ IdP federation live; the approver role maps to `custom:hcls_role` ; a non-approver is correctly **denied** the high-risk tool.
 - ☐ `CONNECTOR_MODE=live` against the real system; a read and a (test) write both succeed and appear in the audit trail.
 - ☐ HITL gate verified: execution pauses; finalize only runs after `send-task-success` with a verified reviewer.
 - ☐ Audit trail is append-only (attempt an update/delete and confirm it's denied); WORM bucket retention set.
 - ☐ `pytest` + `governance` evals captured as **validation evidence**; prompt manifest matches.
 - ☐ Customer CSV/CSA package signed for the intended use; penetration test scheduled/done.
 - ☐ Runbooks reviewed with ops: `runbooks/INCIDENT-RESPONSE.md` , `DR-RUNBOOK.md` , `HITL-QUEUE-OPERATIONS.md` , `MODEL-DEGRADATION-RESPONSE.md` .
-

11. Per-Agent Appendix

Every agent deploys with the **same steps** — change only the values below. `AgentId` is the CloudFormation parameter; the native code/zip lives under `aws-native-reference/<AgentId>/`.

Agent	AgentId	Systems to connect (env * BASE_URL)	High-risk tools (need approval)	Approver role
01 Regulatory Writing	01-regulatory-writing	RIM, DMS	<code>rim.create_submission_draft</code> , <code>dms.put_draft</code>	REGULATORY_A
02 Pharmacovigilance	02-pharmacovigilance	Safety (SAFETY_BASE_URL), MedDRA, WHODrug	<code>safety.write_case_draft</code> , <code>safety.submit_report</code>	PV_MEDICAL_F
03 Clinical Trial Ops	03-clinical-trial-ops	CTMS, eTMF, EDC	<code>edc.create_query</code>	CLINOPS_LEAD
04 Site & Patient Matching	04-site-patient-matching	RWD, CTMS	(reads/cohort; no irreversible writes)	SITE_SELECTI
05 Quality / CAPA	05-quality-capa	QMS	<code>qms.create_capa_draft</code> , <code>qms.close_capa</code>	QUALIFIED_PE
06 Protocol Design	06-protocol-design	RIM (guidance), RWD, CTMS	(drafting; human-approved finalize)	CLINICAL_SC MEDICAL_REVI
07 RWE / HEOR	07-rwe-heor	RWD	(analysis; human-approved synthesis)	EPIDEMIOLOGI
08 Medical Affairs / MSL	08-medical-affairs-msl	CRM, DMS	<code>mlr.submit_for_review</code>	MEDICAL_AFFA

For each, the Step Functions state machine is `hcls-<env>-<AgentId>`, the secret name is `hcls/<system>_api_token`, and the per-agent native rebuild + `DEPLOY.md` live in `aws-native-reference/<AgentId>/`.

12. Teardown (dev)

```
aws cloudformation delete-stack --stack-name hcls-dev-02-pharmacovigilance
```

Note: the audit DynamoDB table and the S3 Object Lock (WORM) bucket are created with **Retain** deletion policies on purpose — regulated records must survive a stack deletion. Remove them deliberately (and only in dev) after exporting any records you need.

13. Troubleshooting

Symptom	Likely cause	Fix
<code>AccessDeniedException</code> calling Bedrock	Model access not granted in this Region	Phase 3.1
Stack fails on <code>AWS::BedrockAgentCore::*</code>	Region doesn't support AgentCore, or property unsupported	Deploy in an AgentCore-enabled Region, or use the API Gateway + Lambda-authorizer + Cognito fallback (see <code>platform_core/.../mcp_gateway/README.md</code>)
LLM factory refuses to start	<code>ENVIRONMENT=prod</code> but no <code>BEDROCK_GUARDRAIL_ID</code>	Set the Guardrail ID (this guard is intentional)
Execution stuck at <code>HumanReviewGate</code>	Working as designed — awaiting human approval	Send <code>send-task-success</code> with a verified reviewer (Phase 8.2)
<code>Safety API call failed</code>	<code>SAFETY_BASE_URL</code> unreachable or token missing	Verify the URL and the <code>hcls/safety_api_token</code> secret; test against the reference service first
Connector raises <code>NotImplementedError</code>	<code>CONNECTOR_MODE=live</code> but that system's live connector isn't wired	Implement the typed methods in <code>connectors/live.py</code> (mirror <code>LiveSafetyConnector</code>)
Tool call denied unexpectedly	User's IdP role lacks the entitlement, or agent isn't granted the tool	Check <code>ROLE_ENTITLEMENTS</code> / <code>AGENT_TOOL_GRANTS</code> in <code>policy.py</code> and the Cognito attribute mapping

This handbook is the engineering companion to `SOLUTION-FIELD-GUIDE.md` (sales/SA), `ENTERPRISE-PLATFORM.md` (architecture), and the per-agent `docs/aws-deployment-guide.md` (summary). For operations after go-live, see `runbooks/`.